Quiz — 2.1.3 Thinking Procedurally

Name: ___ _ Date: _ ***Mark:*** ___ / 20

OCR H446 · Component 02 · 2.1.3

Section A — Multiple choice (6 marks)

1. Breaking the task "build a shopping website" into "user accounts", "product catalogue", "basket" and "checkout" sub-problems is an example of: A. Abstraction B. Decomposition C. Caching D. A race condition Your answer: ☐

2. In a "make a cup of tea" procedure, which step *must* come before "pour water onto the teabag"? A. Add milk B. Boil the water C. Drink the tea D. Wash the cup afterwards Your answer: ☐

3. A self-contained block of code that performs one named task (e.g. `calculateVAT`) and can be called from several places is best described as a: A. Precondition B. Sub-procedure C. Cache D. Decision point Your answer: ☐

4. A robot must "navigate to the box" before it can "pick up the box". This dependency means the two steps must be: A. Run concurrently B. Run in a fixed order C. Abstracted away D. Cached Your answer: ☐

5. Why is decomposition useful when several programmers build one large system together? A. It guarantees the program runs on multiple cores B. Each sub-problem can be assigned to and worked on by a different person C. It removes all the inputs and outputs D. It makes data automatically sorted Your answer: ☐

6. In a payroll program the order is: read hours → calculate gross pay → deduct tax → output payslip. "Deduct tax" must follow "calculate gross pay" because: A. The two steps are independent B. Tax is calculated from the gross pay produced by the earlier step C. Tax can be cached D. The payslip is an input Your answer: ☐

Section B — Short answer (9 marks)

7. A developer is building a vending machine program. Identify **three** sensible sub-procedures the problem could be decomposed into. **[3]**

8. For an ATM "withdraw cash" task, state **two** steps that must happen in a fixed order and explain why one depends on the output of the other. **[2]**

9. Explain **two** advantages of dividing a large program into sub-procedures rather than writing it as one long block of code. **[2]**

10. A quiz game program calls the same `shuffleQuestions` sub-procedure at the start of every round. Explain why writing this as a sub-procedure is better than copying the shuffling code into each round. **[2]**

Section C — Applied question (5 marks)

11. A school wants a program that produces end-of-term reports for every pupil. Using *thinking procedurally*, decompose this task into named sub-procedures and put them in the correct order. For at least one pair of steps, explain why the order is fixed (i.e. one step needs the output of an earlier step). **[5]**

Answer key

Section A (6 marks — 1 each) 1. **B** — splitting one big problem into smaller sub-problems is decomposition. 2. **B** — water must be boiled before it can be poured; ordering with a dependency. 3. **B** — a named, self-contained, callable block of code is a sub-procedure (subroutine). 4. **B** — picking up depends on having navigated there, so the steps are sequential/fixed order. 5. **B** — decomposition lets sub-problems be shared out among different developers. 6. **B** — the tax step consumes the gross pay output by the previous step, so it must follow it.

Section B (9 marks)

7. **[3]** Any three (1 each): accept/validate coins or card payment; let user select a product; check stock/availability; dispense the item; give change; update stock count; display messages. Must be plausible sub-tasks of the vending scenario.

8. **[2]** Any valid ordered pair (1): e.g. "check the balance is sufficient" must come before "dispense the cash"; or "verify the PIN" before "allow withdrawal". Dependency explained (1): the later step relies on the result/output of the earlier step (you cannot safely dispense until the balance check has confirmed funds).

9. **[2]** Any two (1 each): easier to write/understand (smaller pieces); reusable — call the same routine in many places; easier to test/debug each part in isolation; can be shared among a team; easier to maintain/update one routine.

10. **[2]** Writing it once as a sub-procedure means it is defined in a single place (1), so it can be reused/called each round and any fix or change is made once and applies everywhere — avoiding duplicated, inconsistent code (1).

Section C (5 marks) 11. **[5]** Up to 5 from: Named sub-procedures, e.g. `readPupilData` → `calculateGrades` / `averageScores` → `generateComments` → `assembleReport` → `printReport` / `emailReport` (1 for sensible decomposition into named routines, 1 for a logical overall order). Identify a dependency, e.g. grades must be calculated before the report can be assembled, and data must be read before grades can be calculated (1). Explain the dependency: the later step uses the *output* of the earlier step (e.g. `assembleReport` needs the grades produced by `calculateGrades`), so the

order is fixed (1). Recognise that some steps could be independent but the dependent ones cannot be reordered (1). Must apply to the reports scenario with named sub-procedures, not give a generic definition.

Total: 6 + 9 + 5 = 20